

EXPERIMENT NO: 7

Title: A program to simulate Booth's multiplication

Name -Shraddha Gajanan Desai

SE COMPS DIV A BATCH -A Rollno-19

Subject: DLCA Lab
Don Bosco Institute of Technology, Kurla
Academic Year 2024-25

EXPERIMENT NO: 7

A program to simulate Booth's multiplication

AIM	Write a program to simulate Booth's multiplication.
LEARNING OBJECTIVE	To implement the operation of the arithmetic unit including the implementation of fixed point multiplication for signed numbers.
LEARNING OUTCOME	Students will be able to write a higher level language code for simulating hardware operation for Booth's multiplication process.
LAB OUTCOME	CSL 302.6: Ability to implement various algorithms for arithmetic operations.
PROGRAM OUTCOME	PO-1, PO5-2, PO8-3, PO9-3, PO12-2, PSO1-2
BLOOM'S TAXONOMY LEVEL	Create
THEORY	Note for students: Students should write theory related to Booth's algorithm and explain how multiplication is implemented in hardware. Append the flowchart and algorithm. Do mention its pros and cons. *Principle:* Booth's algorithm is based on the principle of repeated addition and shifting. It multiplies two numbers by repeatedly adding or subtracting the multiplicand (the number being multiplied) to/from the product, depending on the multiplier's (the number by which we multiply) bits. *Hardware Implementation:* Booth's algorithm is implemented in hardware using shift registers, adders, and logic gates *Pros:*

	1. Fast multiplication: Booth's algorithm is faster than other multiplication techniques. 2. Low hardware complexity: Requires fewer logic gates and shift registers. 3. Simple implementation: Easy to understand and implement. *Cons:* 1. Limited precision: Suitable for small-precision arithmetic. 2. Dependency on multiplier: Performance depends on the number of 1s in the multiplier. *Applications:* 1. Digital signal processing 2. Microprocessors 3. Embedded systems 4. Cryptography
SOFTWARE USED	C/C++/Java/Python
STEPS TO EXECUTE THE PROGRAM	1. Take two decimal numbers from the user in the range of 0 to 15, both positive and negative (M=Multiplicand & Q=Multiplier) and convert it to 4-bit binary numbers. 2. Negative numbers are represented in two's complement form. 3. Initialize the counter with the count of number of bits. 4. Initialize the one bit register Q₁ as 0. 5. Initialize A to zero, where A is accumulator which stores the MSB of the result. Multiplier Q stores the LSB of the result. 6. Check LSB bit of Q and Q₋₁: a. If they are 11 or 00 arithmetic shift is done for A, Q and Q₋₁ and decrement the counter. b. If they are 01 add A to M and then arithmetic shift is done for A, Q and Q₋₁ and decrement the counter. c. If they are 10 sub M from A and then arithmetic shift is done for A, Q and Q₋₁ and decrement the counter. 7. Check the counter, if it is not 0, move to step 6, otherwise store the result in A and Q.
CODE	Students should append their code. <pre> #include <stdio.h> #include <string.h> void binaryAddition(int A[], int M[], int n) { int carry = 0; for (int i = n - 1; i >= 0; i--) { int sum = A[i] + M[i] + carry; A[i] = sum % 2; carry = sum / 2; } } </pre>

```

    }
}

void arithmeticRightShift(int A[], int Q[], int Q_1[], int n) {
    Q_1[0] = Q[n - 1];
    for (int i = n - 1; i > 0; i--) {
        Q[i] = Q[i - 1];
    }
    Q[0] = A[n - 1];
    for (int i = n - 1; i > 0; i--) {
        A[i] = A[i - 1];
    }
}

void twosComplement(int M[], int n) {
    int one[n];
    for (int i = 0; i < n; i++) one[i] = 0;
    one[n - 1] = 1;
    for (int i = 0; i < n; i++) M[i] = !M[i];
    binaryAddition(M, one, n);
}

void printRegisters(int A[], int Q[], int Q_1[], int step, int n, const char* operation) {
    printf("Step: %d | Operation: %s\n", step, operation);
    printf("A: ");
    for (int i = 0; i < n; i++) printf("%d", A[i]);
    printf("\tQ: ");
    for (int i = 0; i < n; i++) printf("%d", Q[i]);
    printf("\tQ_1: %d\n", Q_1[0]);
}

void boothAlgorithm(int M[], int Q[], int n, int step) {
    int A[n];
    for (int i = 0; i < n; i++) A[i] = 0;
    int Q_1[1] = {0};
    step = 0;

    printf("Initial values:\n");
    printRegisters(A, Q, Q_1, step, n, "Initial");

    while (step < n) {
        if (Q[n - 1] == 1 && Q_1[0] == 0) {
            printRegisters(A, Q, Q_1, step, n, "Subtract M");
            int M_neg[n];
            memcpy(M_neg, M, n * sizeof(int));
            twosComplement(M_neg, n);
            binaryAddition(A, M_neg, n);
        } else if (Q[n - 1] == 0 && Q_1[0] == 1) {
            printRegisters(A, Q, Q_1, step, n, "Add M");
            binaryAddition(A, M, n);
        }
        step++;
    }
}

```

```

    } else {
        printRegisters(A, Q, Q_1, step, n, "No Operation");
    }
    arithmeticRightShift(A, Q, Q_1, n);
    printRegisters(A, Q, Q_1, step + 1, n, "ARS");
    (step)++;

    int choice;
    printf("1. Continue\n2. Exit\n");
    printf("Enter your choice: ");
    scanf("%d", &choice);

    if (choice == 2) {
        break;
    }
}

if (step == n) {
    printf("Final Result: ");
    for (int i = 0; i < n; i++) printf("%d", A[i]);
    for (int i = 0; i < n; i++) printf("%d", Q[i]);
    printf("\n");

    if (A[0] == 1) {
        int result[n * 2];
        for (int i = 0; i < n; i++) result[i] = A[i];
        for (int i = 0; i < n; i++) result[n + i] = Q[i];
        printf("Since the result is negative, converting to two's complement...\n");
        twosComplement(result, n * 2);
        printf("Two's complement of the result: ");
        for (int i = 0; i < n * 2; i++) printf("%d", result[i]);
        printf("\n");
    }
}
}

void toBinary(int num, int binary[], int n) {
    if (num < 0) {
        num = (1 << n) + num; // Convert to two's complement representation
    }

    for (int i = n - 1; i >= 0; i--) {
        binary[i] = num & 1;
        num >>= 1;
    }
}

void main()

```

```

{
    int n;
    int ch;
    int step;
    printf("Enter the number of bits: ");
    scanf("%d", &n);

    int M[n], Q[n];
    int multiplicand, multiplier;

    printf("Enter the multiplicand: ");
    scanf("%d", &multiplicand);
    printf("Enter the multiplier: ");
    scanf("%d", &multiplier);

    toBinary(multiplicand, M, n);
    toBinary(multiplier, Q, n);

    do{
        printf("1Continue\n2Exit\n");
        printf("Enter your choice");
        scanf("%d", &ch);
        if(ch==1)
        {
            printf("Booth's Algorithm Steps:\n");
            boothAlgorithm(M, Q, n, ch);
        }

    }while(ch!=2);

}

```

OUTPUT

Students should append their output showing all cases.

```
Enter the multiplicand: 7
Enter the multiplier: -3
1Continue
2Exit
Enter your choice1
Booth's Algorithm Steps:
Initial values:
Step: 0 | Operation: Initial
A: 0000 Q: 1101 Q_1: 0
Step: 0 | Operation: Subtract M
A: 0000 Q: 1101 Q_1: 0
Step: 1 | Operation: ARS
A: 1100 Q: 1110 Q_1: 1
```

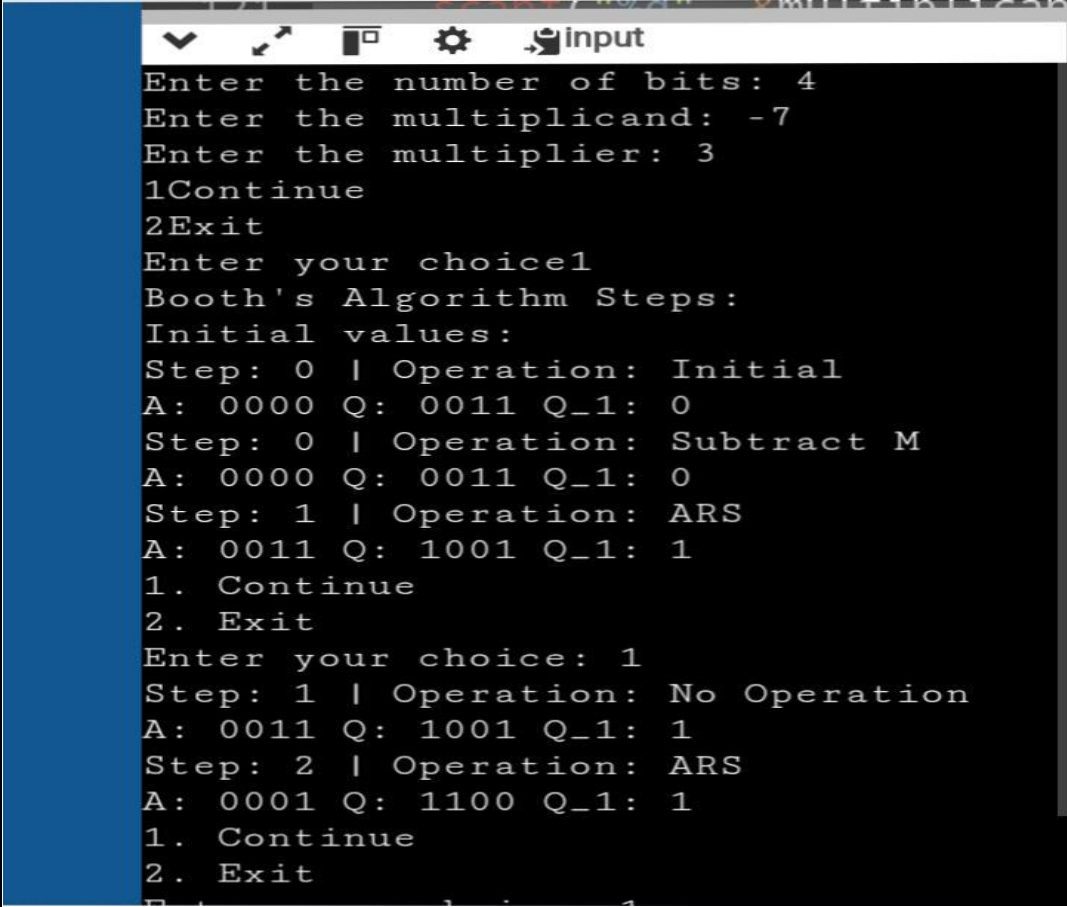
```
1. Continue
2. Exit
Enter your choice: 1
Step: 1 | Operation: Add M
A: 1100 Q: 1110 Q_1: 1
Step: 2 | Operation: ARS
A: 0001 Q: 1111 Q_1: 0
1. Continue
2. Exit
Enter your choice: 1
Step: 2 | Operation: Subtract M
A: 0001 Q: 1111 Q_1: 0
Step: 3 | Operation: ARS
A: 1101 Q: 0111 Q_1: 1
```

```

Enter your choice: 1
Step: 3 | Operation: No Operation
A: 1101 Q: 0111 Q_1: 1
Step: 4 | Operation: ARS
A: 1110 Q: 1011 Q_1: 1
1. Continue
2. Exit
Enter your choice: 1
Final Result: 11101011
Since the result is negative, converting to two's complement...
Two's complement of the result: 00010101
1Continue
2Exit
Enter your choice: 2

...Program finished with exit code 0
Press ENTER to exit console.

```



```

Enter the number of bits: 4
Enter the multiplicand: -7
Enter the multiplier: 3
1Continue
2Exit
Enter your choice: 1
Booth's Algorithm Steps:
Initial values:
Step: 0 | Operation: Initial
A: 0000 Q: 0011 Q_1: 0
Step: 0 | Operation: Subtract M
A: 0000 Q: 0011 Q_1: 0
Step: 1 | Operation: ARS
A: 0011 Q: 1001 Q_1: 1
1. Continue
2. Exit
Enter your choice: 1
Step: 1 | Operation: No Operation
A: 0011 Q: 1001 Q_1: 1
Step: 2 | Operation: ARS
A: 0001 Q: 1100 Q_1: 1
1. Continue
2. Exit

```



```

Enter your choice: 1
Step: 2 | Operation: Add M
A: 0001 Q: 1100 Q_1: 1
Step: 3 | Operation: ARS
A: 1101 Q: 0110 Q_1: 0
1. Continue
2. Exit
Enter your choice: 1
Step: 3 | Operation: No Operation
A: 1101 Q: 0110 Q_1: 0
Step: 4 | Operation: ARS
A: 1110 Q: 1011 Q_1: 0
1. Continue
2. Exit
Enter your choice: 1
Final Result: 11101011
Since the result is negative, converting to two's complement...
Two's complement of the result: 00010101
1Continue
2Exit

```

```

3
Enter the number of bits: 4
Enter the multiplicand: -7
Enter the multiplier: -3
1Continue
2Exit
Enter your choice: 1
Booth's Algorithm Steps:
Initial values:
Step: 0 | Operation: Initial
A: 0000 Q: 1101 Q_1: 0
Step: 0 | Operation: Subtract M
A: 0000 Q: 1101 Q_1: 0
Step: 1 | Operation: ARS
A: 0011 Q: 1110 Q_1: 1
1. Continue
2. Exit
Enter your choice: 1
Step: 1 | Operation: Add M
A: 0011 Q: 1110 Q_1: 1
Step: 2 | Operation: ARS
A: 1110 Q: 0111 Q_1: 0

```

	<pre> Enter your choice: 1 Step: 1 Operation: Add M A: 0011 Q: 1110 Q_1: 1 Step: 2 Operation: ARS A: 1110 Q: 0111 Q_1: 0 1. Continue 2. Exit Enter your choice: 1 Step: 2 Operation: Subtract M A: 1110 Q: 0111 Q_1: 0 Step: 3 Operation: ARS A: 0010 Q: 1011 Q_1: 1 1. Continue 2. Exit Enter your choice: 1 Step: 3 Operation: No Operation A: 0010 Q: 1011 Q_1: 1 Step: 4 Operation: ARS A: 0001 Q: 0101 Q_1: 1 1. Continue 2. Exit Enter your choice: 1 Final Result: 00010101 </pre>
CONCLUSION	To be written by students Implemented various cases for Booths Multiplication Algorithm
REFERENCES	1. William Stallings, "Computer Organization and Architecture: Designing for Performance", Pearson Publication, 10 th Edition, 2013

Class: S.E Comps (Sem III)

Lecturer: Sejal M Chopra

Subject: DLCA Lab
Don Bosco Institute of Technology, Kurla
Academic Year 2024-25

	2. B. Govindarajulu, "Computer Architecture and Organization: Design Principles and Applications", Second Edition, McGraw Hill (India)
--	---

Class: S.E Comps (Sem III)

Lecturer: Sejal M Chopra

Subject: DLCA Lab